

Hardware-Aware Semantic Segmentation Engine for On-Chip Retinal Prosthesis Navigation

Arnav Agarwal*, Daksh Sawke*, Laxmeesha Somappa
 Department of Electrical Engineering, IIT Bombay, Mumbai, India
 Email: laxmeesha@ee.iitb.ac.in

Abstract—Epiretinal prostheses restore partial vision to patients with photoreceptor degeneration, yet current systems provide only brightness mapping with no semantic scene understanding, limiting navigation utility. This paper presents NanoSeg, a hardware-aware semantic segmentation engine targeting a spectacle-mounted vision processing unit (VPU) for retinal prostheses. The model classifies each pixel of a 60×80 preprocessed single-channel input into four navigation-relevant classes (background, traversable, obstacle, hazard) and was developed through seven documented iterative architecture refinements on the SANPO-Real dataset, with each design decision driven by on-chip resource constraints. NanoSeg contains 12,616 parameters (12.6 KB under INT8 quantization), and a streaming buffer analysis estimates a total on-chip memory footprint of 28.8 KB. Despite being $29\times$ (vs. ENet) to $873\times$ (vs. DeepLabV3+) smaller than evaluated baselines, NanoSeg achieves mIoU 0.464 and the highest mIoU-per-million parameters efficiency among all evaluated architectures. FPGA C-synthesis on the Xilinx Zynq-7020 via Vitis HLS confirms 67.2 ms inference latency at 100 MHz with 46% BRAM and 66% LUT utilization, demonstrating feasibility within wearable hardware budgets.

Index Terms—epiretinal prosthesis, semantic segmentation, streaming architecture, hardware-aware neural network, FPGA, post-training quantization, neural interface

I. INTRODUCTION

Retinal degenerative diseases such as retinitis pigmentosa (RP) and age-related macular degeneration (AMD) progressively destroy photoreceptor cells while leaving the inner retinal circuitry largely intact. AMD alone affects an estimated 196 million people worldwide, and is projected to reach 288 million by 2040 [1], while RP affects approximately 1.5 million individuals [2]. Together, these conditions represent the leading causes of irreversible vision loss.

Epiretinal prostheses offer a promising intervention by stimulating residual retinal ganglion cells with spatially encoded electrical pulses, generating phosphene percepts that restore coarse functional vision. The Argus II system (Second Sight Medical Products), the most widely deployed clinical device, demonstrated meaningful mobility improvements with a 60-electrode array [3]. However, its belt-worn VPU reduces camera input to low-resolution brightness maps [3], limiting semantic understanding, motivating next-generation prostheses with integrated deep-learning-based navigation assistance.

This paper presents NanoSeg, a hardware-aware semantic segmentation engine targeting a spectacle-mounted VPU for retinal prostheses. Our contributions are:

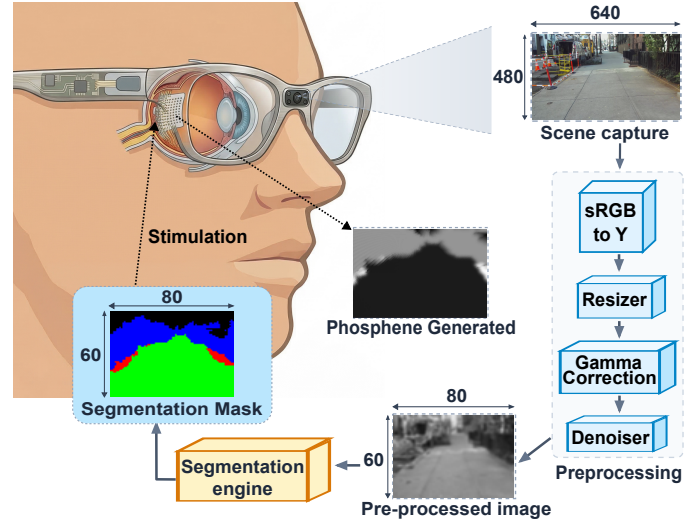


Fig. 1. Proposed spectacle-mounted VPU pipeline. A 640×480 sRGB camera frame is converted to a 60×80 single-channel input, segmented into four navigation classes, and encoded for electrode stimulation.

- A custom CNN architecture, *NanoSeg*, with 12.6 KB of weight storage under INT8 quantization, for four-class navigation segmentation in retinal prostheses.
- A systematic design exploration across 7 architectural variants, documenting the trade-offs between segmentation accuracy, model capacity, and on-chip resource consumption.
- FPGA resource utilization on the Xilinx Zynq-7020 via Vitis HLS C-synthesis, confirming feasibility within wearable-class hardware budgets.

II. RELATED WORK

Beyond the Argus II, other retinal prosthesis approaches include the Alpha AMS subretinal implant [4] and the PRIMA photovoltaic array [5]. These systems relay brightness or contrast information to the electrode array without pixel-level semantic understanding.

Recent work has explored deep learning as a preprocessing strategy for prosthetic vision. Han et al. [6] showed that semantic segmentation significantly outperforms saliency and depth-based approaches for scene understanding in simulated prosthetic vision, using standard models (DeepLab, DeepGaze II) evaluated in software. Rollend et al. [7] developed CNN-based face detection and object recognition algorithms for the Argus II. Both works validate the utility of deep learning for prosthetic vision, but operate on general-purpose hardware.

*These authors contributed equally to this work.

The authors gratefully acknowledge the financial support provided by MeitY, Government of India.

Translating these to wearable devices requires highly constrained edge computing. Lightweight segmentation architectures such as ENet [8] (370 K params), CGNet [9] (0.5 M), and Fast-SCNN [10] (1.1 M) target real-time GPU inference. Hardware-aware co-design efforts, including MCUNet [11] and hls4ml-based FPGA segmentation by Ghielmetti et al. [12] push toward edge deployment, but still require hundreds of kilobytes of storage. Attempting to reduce this footprint via pruning at >97% compression offers no accuracy advantage over training compact architectures from scratch [13] and complicates streaming hardware design.

III. DESIGN CONSTRAINTS

Latency and safety: At pedestrian speeds, a 100 ms delay corresponds to ~ 15 cm of unguided travel. Missed hazard detection endangers the user, requiring sub-100 ms inference.

Power and form factor: A spectacle-mounted low-power device must sustain multi-hour operation with a lightweight battery, restricting the design to fixed-point arithmetic and on-chip-only memory access.

Memory and input representation: All weights, activations, and buffers must reside in on-chip SRAM with no external DRAM. A single-channel 60×80 input is used to minimize first-layer compute and memory footprint.

IV. DATASET AND PREPROCESSING

A. Dataset

Models are trained and evaluated on the SANPO-Real dataset [14], which provides egocentric video with human-annotated semantic masks, closely matching a spectacle-mounted viewpoint. We use 40 manually annotated sessions, yielding 3,386 training and 598 test images. Original annotations are mapped to four navigation-relevant classes: *background* (sky, walls), *traversable* (walkable surface), *obstacle* (static/dynamic obstruction), and *hazard* (stairs, inaccessible surface, water body).

The test set is imbalanced: Background 40.8%, Traversable 34.6%, Obstacle 22.2%, Hazard 2.4%. The Hazard class is $\sim 17\times$ rarer than Background, motivating aggressive class weighting due to its safety-critical nature.

B. Preprocessing Pipeline

To ensure the model is trained on inputs representative of the target hardware pipeline, all dataset images are passed through the same preprocessing stages that the VPU would apply at inference time. As illustrated in Fig. 1, a 640×480 sRGB camera frame passes through four stages before reaching the segmentation model: (i) conversion to a single luminance channel (ITU-R BT.601), (ii) spatial downsampling to 60×80 via area averaging, (iii) gamma correction ($\gamma=2.2$) to restore the perceptual luminance distribution, and (iv) a 3×3 smoothing filter to suppress downsampling artifacts.

V. MODEL ARCHITECTURE AND DESIGN ITERATIONS

The final architecture, NanoSeg (V7), reached after seven stages of refinement, is shown in Fig. 2 and all the architectural variants (V1 to V7) are briefly described in this section.

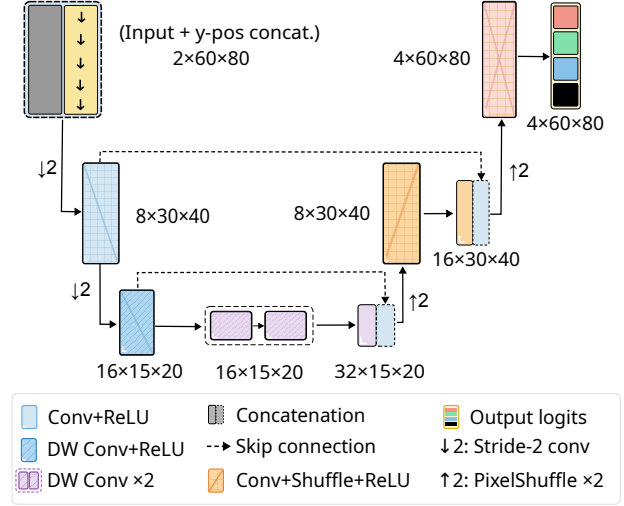


Fig. 2. Customized NanoSeg (V7) architecture: Dual skip connections (enc1 $\rightarrow$ dec2, enc2 $\rightarrow$ dec1) provide U-Net-style multi-scale feature reuse. Block height encodes spatial resolution. Vertical position encoding is concatenated at the input.

A. Loss Functions

All architectural variants use weighted cross-entropy

$$\mathcal{L}_{CE} = - \sum_c w_c y_c \log \hat{p}_c, \quad \mathbf{w} = [0.3, 0.25, 0.7, 3.0], \quad (1)$$

where weights correspond to background, traversable, obstacle, and hazard. Selected variants augment (1) with a smoothness regulariser penalising abrupt class transitions,

$$\mathcal{L}_{smooth} = \frac{\lambda}{N} \sum_{i,j} [(f_{i,j+1} - f_{i,j})^2 + (f_{i+1,j} - f_{i,j})^2], \quad \lambda=0.1, \quad (2)$$

giving $\mathcal{L} = \mathcal{L}_{CE} + \mathcal{L}_{smooth}$. One ablation substitutes focal loss [15],

$$\mathcal{L}_{FL} = - \sum_c w_c (1 - \hat{p}_c)^\gamma \log \hat{p}_c, \quad \gamma=2, w_3=4.5, \quad (3)$$

labelled V7_F.

B. Initial Architecture

The architecture family, shown in Fig. 2 in its final form, follows a streaming encoder-decoder with PixelShuffle upsampling [16]. The encoder comprises two strided Conv-Batch Normalisation (BN)-ReLU blocks: **enc1** ($60 \times 80 \rightarrow 30 \times 40$, 8 channels) and **enc2** ($30 \times 40 \rightarrow 15 \times 20$, 16 channels), followed by a depthwise (DW, groups = channels) bottleneck. The decoder mirrors this with two PixelShuffle upsampling stages: **dec1** ($15 \times 20 \rightarrow 30 \times 40$) and **dec2** ($30 \times 40 \rightarrow 60 \times 80$), each preceded by a $C \rightarrow 4C$ point-wise expansion.

PixelShuffle is chosen over transpose convolution because it avoids overlap in output computation, making it compatible with a row-sequential streaming datapath and avoiding the output-side buffering required by transpose convolution. Full depthwise convolutions reduce the bottleneck MAC count $16\times$ at the cost of zero inter-channel mixing below enc1. Loss: (1). Parameters: 6,136; mIoU: 0.429.

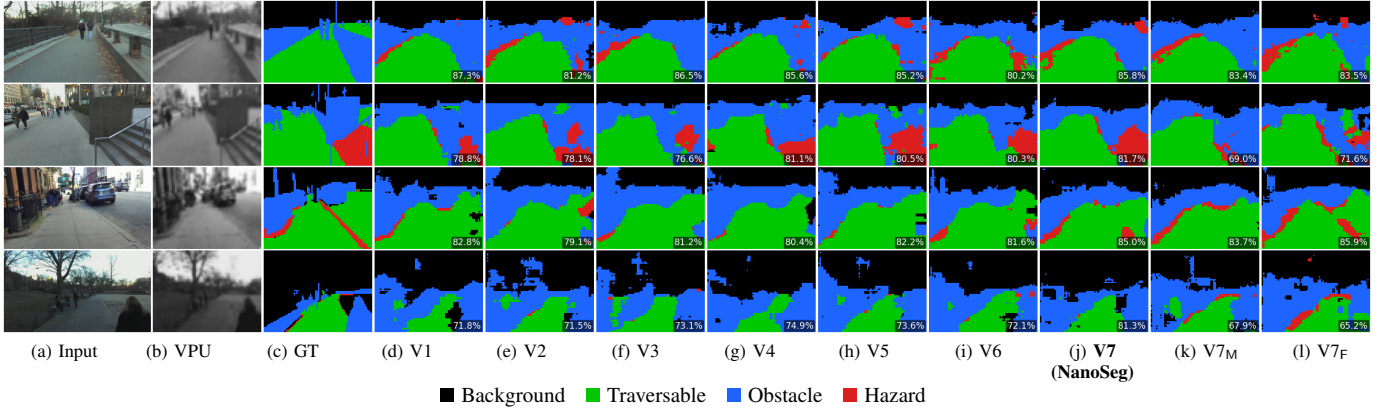


Fig. 3. Qualitative comparison across all model versions on four representative scenes. (a) input 640×480 ; (b) VPU-processed 60×80 ; (c) ground truth (GT); (d)–(j) predictions of V1 through V7 (proposed final); (k) V7_M: 80-session training with manual class weights; (l) V7_F: focal loss ablation.

TABLE I

ITERATIVE MODEL COMPARISON ON SANPO TEST SET. PER-CLASS COLUMNS REPORT IOU; HAZARD RECALL (HAZD-R) IS DEFINED AS IN (5).

Ver.	Architecture Change	mIoU	Backg	Trav	Obst	Hazd	Hazd-R	Params
V1	Initial: full-DW enc-dec, PixelShuffle	0.429	0.532	0.609	0.382	0.194	0.462	6,136
V2	+ Shadow augmentation	0.390	0.485	0.561	0.359	0.154	0.464	6,136
V3	+ Smoothness loss + shadow augmentation	0.402	0.492	0.578	0.367	0.173	0.428	6,136
V4	+ Smoothness loss only	0.430	0.521	0.606	0.377	0.218	0.466	6,136
V5	+ Skip enc1→dec2, relaxed DW grouping	0.433	0.535	0.614	0.377	0.208	0.474	7,576
V6	+ Vertical position encoding	0.438	0.549	0.619	0.382	0.201	0.455	7,648
V7 (NanoSeg)	+ Dual skip (enc2→dec1), deeper bottleneck	0.464	0.591	0.640	0.404	0.220	0.530	12,616
V7 _M	V7 + 80-session training, manual weights	0.418	0.601	0.627	0.358	0.087	0.106	12,616
V7 _F	V7 + 80-session training, focal loss	0.403	0.586	0.599	0.348	0.078	0.233	12,616

C. Iterative Architecture Refinements

Seven incremental refinements are summarised in Table I; Fig. 3 shows qualitative outputs across all versions on 4 held-out scenes. For each change we report the accuracy gain (Δ mIoU) and the estimated additional activation memory under a streaming execution model (detailed in Section VI).

V2 [Shadow Augmentation]: Random shadow masks were applied to improve lighting robustness. Loss: (1); mIoU fell by 0.039 from V1 — the model lacked capacity to absorb the augmentation distribution shift.

V3 [Smoothness Loss (+ Shadow Augmentation)]: Loss: $\mathcal{L}_{CE} + \mathcal{L}_{smooth}$. Smoothness regularisation partially recovered accuracy (Δ mIoU = +0.012 over V2) but did not surpass V1, confirming augmentation as the primary degradation source.

V4 [Smoothness Loss Only]: Loss: $\mathcal{L}_{CE} + \mathcal{L}_{smooth}$; no augmentation. mIoU nearly matched V1 (0.429) while Hazard IoU improved. The smoothness penalty suppresses spatially fragmented predictions, particularly beneficial for the rare Hazard class, where isolated false positives dominate the error term.

V5 [Skip Connection and Relaxed Channel Mixing]: Loss unchanged from V4 ($\mathcal{L}_{CE} + \mathcal{L}_{smooth}$). A skip from enc1 to dec2 concatenates multi-scale features into the final upsampling stage; this is streaming compatible: part of the enc1 map ($30 \times 40 \times 8 = 9,600$ B INT8) is buffered until dec2 reaches the corresponding position. Grouping was relaxed in enc2 and the bottleneck, restoring inter-channel mixing at modest parameter

cost. Δ mIoU = +0.003 over V4. Memory overhead: +9.4 KB. **V6 [Vertical Position Encoding]:** Loss unchanged from V4. A precomputed channel $y_{pos} \in [0, 1]$ is concatenated to the grayscale input, providing a static geometric prior: identical luminance at the top of the frame predicts background (sky) while at the bottom it predicts traversable (ground shadow), resolving a systematic ambiguity in the grayscale domain. Δ mIoU = +0.005 over V5. Memory overhead: 0 B.

V7 [Dual Skip Connections and Deeper Bottleneck (Final)]: Loss unchanged. A second skip completes full U-Net-style multi-scale feature reuse. The bottleneck is deepened to two grouped convolutions, increasing capacity at the most compressed scale where the receptive field is largest. Δ mIoU = +0.026 over V6; all four per-class IoUs improve. Memory overhead: +4.7 KB SRAM. This architecture is Pareto-optimal: no other versions achieve both >0.45 mIoU and >0.20 Hazard IoU (Table I).

Extended ablations: Training V7 on 80 sessions with manual class weights collapsed Hazard IoU to 0.087. The enlarged dataset diluted the hazard-region supervision below the threshold where weighted-CE can compensate. Substituting focal loss (3) ($\gamma=2$, $w_3=4.5$) similarly failed (Hazard IoU 0.078), as down-weighting easy negatives conflicts with the extreme class imbalance at 60×80 resolution.

TABLE II
ON-CHIP MEMORY ESTIMATE AND FPGA RESOURCE UTILIZATION
(ZYNQ-7020, 100 MHz)

Component	Size	Resource	Used	Utilization
Weights	12.6 KB	BRAM	130	46%
Scales	0.2 KB	DSP	6	2%
Line Buffer	1.9 KB	FF	6.8k	6%
Skip Buffer	14.1 KB	LUT	35k	66%
Total	28.8 KB	Inference Latency	67.2 ms	

VI. IMPLEMENTATION AND RESULTS

All models are implemented in PyTorch with Adam [17], using a learning rate 10^{-3} decayed via cosine annealing to 10^{-5} over 300 epochs, with batch size 32 and no test-time augmentation. All weights are quantized to INT8 (symmetric per-channel, INT16 accumulators). The estimated on-chip footprint is 28.8 KB (Table II, left): convolutional layers use three-row line buffers, while skip connections retain full feature maps until consumed by their respective decoder stages. For FPGA C-synthesis on the Zynq-7020 at 100 MHz, PixelShuffle was replaced with nearest-neighbour upsampling and BN folded into convolutional biases. Results (Table II, right) confirm 67.2 ms inference latency with 66% LUT and 46% BRAM utilization; low FF usage (6%) indicates a memory-bound design.

A. Evaluation Metrics

We report mean Intersection-over-Union (mIoU) and per-class IoU as primary accuracy metrics:

$$\text{mIoU} = \frac{1}{C} \sum_c \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}. \quad (4)$$

where C denotes the total number of semantic classes. Additionally, **Hazard Recall (Hazard-R)** — the fraction of ground-truth Hazard pixels correctly detected — is reported as a safety metric:

$$R_{\text{Hz}} = \frac{\text{TP}_{\text{Hz}}}{\text{TP}_{\text{Hz}} + \text{FN}_{\text{Hz}}}. \quad (5)$$

B. Iterative Model Results

Table I reports all metrics across the design trajectory. Excluding the unsuccessful augmentation experiments (V2–V3), mIoU improves monotonically from V1 to V7, while Hazard Recall rises from 0.462 to 0.530. The 80-session ablations collapse this metric, validating the 40-session weighted-CE configuration. Fig. 3 shows qualitative outputs across scenes.

C. Comparison with State-of-the-Art

Table III compares NanoSeg against four models fine-tuned on identical pre-processed 60×80 input. General-purpose models achieve higher mIoU (0.631–0.742 vs. 0.464) due to their $29 \times$ – $873 \times$ larger size, but none fit within the sub-15KB weight budget. NanoSeg achieves 36.78 mIoU per million parameters, $21 \times$ higher than ENet (1.71), demonstrating that architecture co-design under ASIC constraints yields efficiency gains far beyond parameter reduction alone.

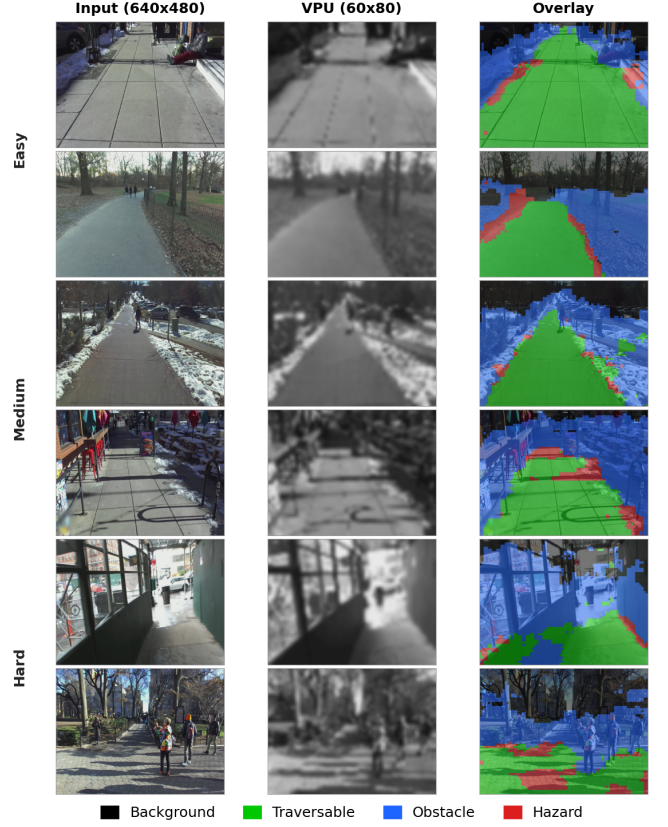


Fig. 4. NanoSeg inference on six outdoor scenes (rows 1–2: easy; 3–4: medium; 5–6: hard). Columns: input 640×480 ; VPU-processed 60×80 (upscaled for display); segmentation overlay (50% blend).

TABLE III
COMPARISON WITH SOTA SEGMENTATION MODELS ON THE SANPO
TEST SET (60×80 INPUT, 598 IMAGES).

Model	Params	mIoU	mIoU/M [†]	Mem	ASIC
SegFormer-B0 [18]	3.72M	0.728	0.20	>14 MB	✗
ERFNet [19]	2.06M	0.742	0.36	>8 MB	✗
DeepLabV3+ [20]	11.0M	0.672	0.06	>42 MB	✗
ENet [8]	0.37M	0.631	1.71	>3 MB	✗
NanoSeg V7 (Ours)	12.6K	0.464	36.78	28.8 KB	✓

[†]mIoU per million params. Baselines pretrained on ADE20K/Cityscapes/ImageNet, finetuned on same split. Baseline memory exceeds ASIC budget by 100 – $1400 \times$.

VII. CONCLUSION

This paper presented NanoSeg, a 12,616-parameter segmentation CNN for four-class navigation in epiretinal prostheses, designed through seven documented iterative architectural refinements under ASIC memory and compute constraints. The model achieves mIoU 0.464 with 12.6 KB INT8 weights, and FPGA C-synthesis on the Zynq-7020 confirms 67.2 ms inference at 100 MHz, validating deployability within wearable hardware budgets. To our knowledge, this is the first work to demonstrate semantic segmentation at the sub-15 KB weight budget required for prosthesis-integrated deployment. Future work targets a custom streaming ASIC in 65 nm CMOS.

REFERENCES

- [1] W. L. Wong, X. Su, X. Li, C. M. G. Cheung, R. Klein, C.-Y. Cheng, and T. Y. Wong, "Global prevalence of age-related macular degeneration and disease burden projection for 2020 and 2040: a systematic review and meta-analysis," *The Lancet Global Health*, vol. 2, no. 2, pp. e106–e116, 2014.
- [2] D. T. Hartong, E. L. Berson, and T. P. Dryja, "Retinitis pigmentosa," *The Lancet*, vol. 368, no. 9549, pp. 1795–1809, 2006.
- [3] L. da Cruz, J. D. Dorn, M. S. Humayun, G. Dagnelie, J. Handa, P.-O. Barale, J.-A. Sahel, P. E. Stanga, F. Hafezi, A. B. Safran, *et al.*, "Five-year safety and performance results from the Argus II retinal prosthesis system clinical trial," *Ophthalmology*, vol. 123, no. 10, pp. 2248–2254, 2016.
- [4] K. Stingl *et al.*, "Interim results of a multicenter trial with the new electronic subretinal implant Alpha AMS in 15 patients blind from inherited retinal degenerations," *Frontiers in Neuroscience*, vol. 11, p. 445, 2017.
- [5] D. Palanker *et al.*, "Photovoltaic restoration of central vision in atrophic age-related macular degeneration," *Ophthalmology*, vol. 127, no. 8, pp. 1097–1104, 2020.
- [6] N. Han, S. Srivastava, A. Xu, D. Klein, and M. Beyeler, "Deep learning-based scene simplification for bionic vision," in *Proc. ACM Augmented Humans (AHs)*, pp. 45–54, 2021.
- [7] D. Rollend, P. Rosendall, S. Billings, P. Burlina, K. Wolfe, and K. Katyal, "Face detection and object recognition for a retinal prosthesis," in *ACCV 2016 Workshops, LNCS*, vol. 10116, pp. 303–313, Springer, 2017.
- [8] A. Paszke, A. Chaurasia, S. Kim, and E. Culurciello, "ENet: A deep neural network architecture for real-time semantic segmentation," *arXiv preprint arXiv:1606.02147*, 2016.
- [9] T. Wu, S. Tang, R. Zhang, J. Cao, and Y. Zhang, "CGNet: A light-weight context guided network for semantic segmentation," *IEEE Transactions on Image Processing*, vol. 30, pp. 1169–1179, 2021.
- [10] R. P. K. Poudel, S. Liwicki, and R. Cipolla, "Fast-SCNN: Fast semantic segmentation network," *arXiv preprint arXiv:1902.04502*, 2019.
- [11] J. Lin, W.-M. Chen, Y. Lin, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Proc. NeurIPS*, 2020.
- [12] N. Ghielmetti *et al.*, "Real-time semantic segmentation on FPGAs for autonomous vehicles with hls4ml," *Machine Learning: Science and Technology*, vol. 3, no. 4, p. 045011, 2022.
- [13] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *Proc. ICLR*, 2019.
- [14] V. Blanchard *et al.*, "Sanpo: A scene understanding, accessibility, navigation, pathfinding, obstacle dataset," *arXiv preprint arXiv:2309.03254*, 2023.
- [15] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE ICCV*, pp. 2980–2988, 2017.
- [16] W. Shi *et al.*, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proc. IEEE CVPR*, pp. 1874–1883, 2016.
- [17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [18] E. Xie *et al.*, "Segformer: Simple and efficient design for semantic segmentation with transformers," in *Proc. NeurIPS*, 2021.
- [19] E. Romera *et al.*, "Erfnet: Efficient residual factorized convnet for real-time semantic segmentation," *IEEE Trans. Intelligent Transportation Systems*, 2017.
- [20] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *Proc. ECCV*, pp. 801–818, 2018.